

MecâniQA Tech: Infraestrutura Containerizada, Orquestrada e Resiliente

Autores: Joaqson Rodrigues Miranda
Lucas Santos Oliveira
Manoel Souza Santos
Monique Prado Pereira Gomes
Raian Rocha Santos

Agenda

O objetivo é apresentar o desafio da MecâniQA Tech e a solução desenvolvida pela equipe para estruturar uma infraestrutura containerizada, orquestrada e resiliente.

1. Contexto do Produto:

MecâniQA Tech e o desafio de escalabilidade.

2. Objetivos da Sprint:

Containerização, orquestração e infraestrutura como código.

3. Especificação Final:

Dockerfiles, Docker Compose, Kubernetes e Terraform.

4. Arquitetura e Validação:

Arquitetura da solução, K9s, comunicação entre serviços e provisionamento com Terraform.

5. Considerações Finais:

Resultados alcançados e consolidação da entrega.

Contexto Inicial do Produto

- SaaS de gestão e diagnóstico para oficinas mecânicas
- Stack atual: Java + MySQL + Redis
- Problema: infraestrutura local com limitações de escalabilidade



Objetivos da Sprint



1

Camada Virtual (Docker)

Dockerfiles isolados para Java, MySQL e Redis



2

Orquestração Local

Docker-compose.yml com rede DNS interna e persistência

3

Infraestrutura como Código

Configuração inicial de provisionamento utilizando Terraform



4

Camada de Controle (K8s)

Pods, Deployments e Services monitorados com K9s

MecâniQA Tech:

Infraestrutura Containerizada, Orquestrada e Resiliente

Especificação do Produto Final



1 Empacotamento de Serviços

Dockerfiles independentes para Java, MySQL e Redis

2 Orquestração e Redes

Docker-compose.yml sobre todo o ecossistema com persistência

3 Orquestração Kubernetes

Manifestos de Deployments e Services para API, MySQL e Redis

4 Provisionamento com Terraform

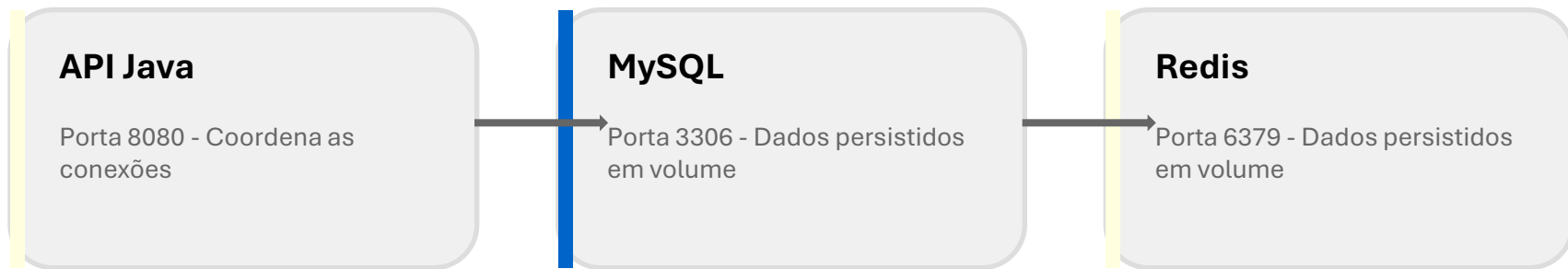
Provider Kubernetes e gerenciamento declarativo do namespace mecaniqa

MecâniQA Tech:

Infraestrutura Containerizada, Orquestrada e Resiliente

Arquitetura da Solução

Três serviços conectados pela rede interna mecaniQA



`depends_on` e `healthchecks` auxiliam no controle da inicialização e na verificação da disponibilidade dos serviços.

Execução local com Docker Compose



O ecossistema sobe de forma integrada e expõe as portas necessárias.

Serviços ativos

```
api -> 8080:8080`nmysql-> 3306:3306`nredis ->
6379:6379
```



A rede mecaniQA permite que os containers encontrem mysql e redis pelos nomes dos serviços.

Monitoramento do Cluster com K9s



Os workloads da aplicação foram implantados e monitorados no cluster Kubernetes.

Pods ativos

- ✓ API: 2/2 Running
- ✓ MySQL: 1/1 Running
- ✓ Redis: 1/1 Running
- ✓ Cluster: docker-desktop

```
Context: docker-desktop [RW]
Cluster: docker-desktop
User: docker-desktop
K9s Rev: v0.51.0
K8s Rev: v1.36.1
CPU: n/a
MEM: n/a
```

NAME	IP	STATUS	RESTARTS	AGE
api-58d9d59788-6w5pd	10.244.0.11	Running	0	9m5s
api-58d9d59788-hrqsl	10.244.0.10	Running	0	14m
mysql-78f8f658c9-bqqt7	10.244.0.7	Running	0	27m
redis-68f6ffbc9-qlsnl	10.244.0.8	Running	0	27m

Resultado: todos os Pods da aplicação estão ativos e disponíveis no cluster.

Mecânica Tech:

Infraestrutura Containerizada, Orquestrada e Resiliente

Validação da Comunicação

Os testes confirmam a conectividade entre a API e os serviços de apoio.

```
moniq@PLANEJAR:~/mecaniQA-fortaleza2$ kubectl exec deploy/api -- getent hosts mysql
10.96.98.225      mysql.default.svc.cluster.local  mysql.default.svc.cluster.local  mysql
moniq@PLANEJAR:~/mecaniQA-fortaleza2$ kubectl exec deploy/api -- getent hosts redis
10.96.179.65     redis.default.svc.cluster.local  redis.default.svc.cluster.local  redis
moniq@PLANEJAR:~/mecaniQA-fortaleza2$ kubectl exec deploy/api -- sh -c 'nc -zv mysql 3306'
mysql (10.96.98.225:3306) open
moniq@PLANEJAR:~/mecaniQA-fortaleza2$ kubectl exec deploy/api -- sh -c 'nc -zv redis 6379'
redis (10.96.179.65:6379) open
moniq@PLANEJAR:~/mecaniQA-fortaleza2$ |
```

Evidências

mysql:3306 -> open`nredis:6379 ->
open`nDNS interno resolvendo
mysql e redis

Resultado: Comunicação validada no ambiente Kubernetes.

Validação da Infraestrutura

A configuração com Terraform foi validada e aplicada ao ambiente Kubernetes.

```
Success! The configuration is valid.

kubernetes_namespace_v1.mecanika: Refreshing state... [id=mecanika]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.
kubernetes_namespace_v1.mecanika
NAME      STATUS    AGE
mecanika  Active    39m
```

Resultados

- ✓ Configuração Terraform válida
- ✓ Provider Kubernetes configurado
- ✓ Namespace mecanika provisionado
- ✓ Estado da infraestrutura sincronizado

Resultado: provisionamento inicial da infraestrutura como código validado com sucesso.

Considerações finais



- ✓ **Entrega:** repositório com Dockerfiles, docker-compose.yml, manifestos K8s e configuração inicial com Terraform
- ✓ **Papel da equipe:** Cloud/DevOps, migrando de infra local instável para base containerizada
- ✓ **Resultado:** ambiente containerizado, orquestrado e com provisionamento inicial definido como código
- ✓ **Validação:** serviços executados e comunicação entre API, MySQL e Redis confirmada nos ambientes implementados.

Referências



- Documentação oficial do Docker — docs.docker.com
- Documentação oficial do Kubernetes — kubernetes.io/docs
- K9s — Kubernetes CLI To Manage Your Clusters In Style — k9scli.io
- Documentação oficial do Terraform — developer.hashicorp.com/terraform